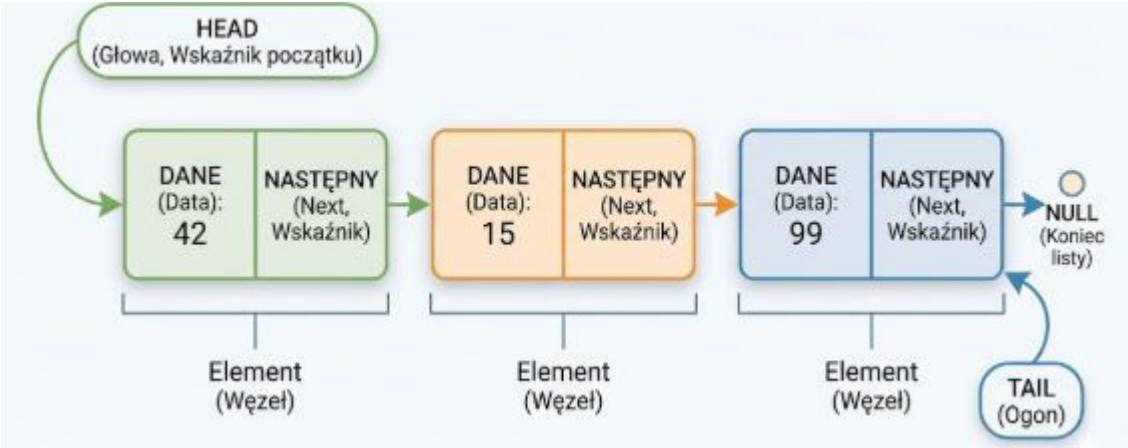
Zaawansowane Struktury Danych

Aleksander Zach, Daniel Kędziora - Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Krakow

Po co nam zaawansowane struktury danych?

Lista “LinkedList”:



Czy nie możemy używać jej zawsze?

Złożoność obliczniowa operacji:

-append O(n) lub O(1)

-delete O(1)

-insert O(n)

-search O(n)

Po co ograniczać się do O(n), jak można lepiej?

Filtr Blooma

Struktura danych

Probabilistyczna struktura, która nie przechowuje oryginalnych danych, a jedynie "zaznacza obecność" za pomocą pojedynczych bitów.

Wydajność

Bardzo szybkie operacje wyszukiwania i dodawania elementów o stałej złożoności czasowej **O(k)**.

Wiarygodność

Możliwe fałszywe pozytywy, ale gwarantowana pewność dla braku elementów (brak fałszywych negatywów).

Ograniczenia

Standardowa implementacja nie oferuje możliwości usuwania elementów po ich dodaniu.

Zastosowania Filtru Blooma

Sprawdzanie przynależności do zbioru

Używamy, gdy chcemy szybko sprawdzić, czy element znajduje się w ogromnej bazie danych (np. czy nazwa użytkownika jest już zajęta) bez kosztownego odpytywania dysku.

Nowoczesne bazy danych

Systemy takie jak Apache Cassandra wykorzystują filtry Blooma do unikania zbędnych odczytów z plików SSTable.

Bezpieczeństwo sieciowe

Przeglądarki internetowe sprawdzają adresy URL pod kątem złośliwego oprogramowania i stron phishingowych.

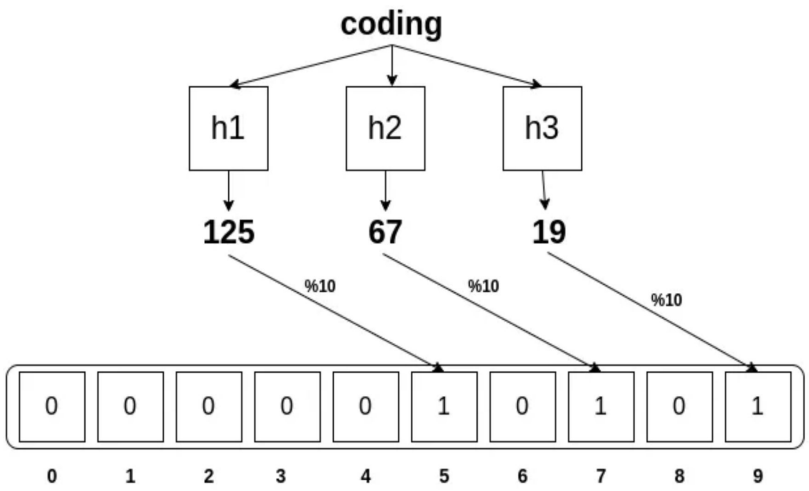
Filtrowanie spamu

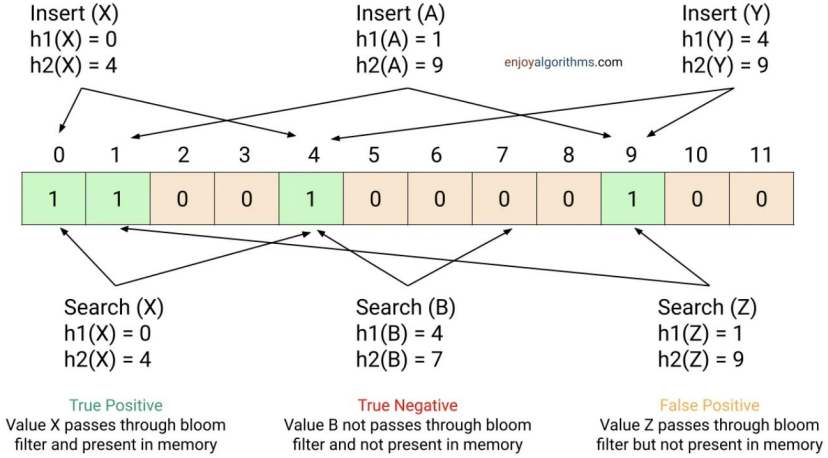
Szybkie odrzucanie wiadomości od znanych spamerów bez pełnej analizy treści maila.

Walidacja haseł

Sprawdzanie, czy nowo wybrane hasło nie znajduje się w słowniku haseł wyciekłych lub zbyt słabych.

Zasady działania





Wzory

m - rozmiar tablicy bitów k - ilość funkcji hash n- przewidywana ilość elementów dodanych do filtra

prawdopodobieństwo wystąpienia fałszywego pozytywu: $P = \left(1 - \left[1 - \frac{1}{m}\right]^{kn}\right)^k$

Rozmiar tablicy: $m = -\frac{n \ln P}{(ln2)^2}$

optymalna ilość funkcji hashujących: $k = \frac{m}{n} ln2$

Funkcje Hashowania

Kontekst i Wyzwanie

- Stosowane funkcje hashujące to np. **murmur**
- **Problem:** Im więcej funkcji, tym większy narzut czasowy.

```python
def pobierz_indeksy_dla_elementu(element, liczba_funkcji_k, rozmiar_tablicy_m):
    glowny_hash = MurmurHash3_64bit(element)
    hash1 = glowny_hash & 0xFFFFFFFF            # dolne 32 bity
    hash2 = (glowny_hash >> 32) & 0xFFFFFFFF    # górne 32 bity
    indeksy = []
    for i in range(1, liczba_funkcji_k + 1):
        obliczony_indeks = (hash1 + i * hash2) % rozmiar_tablicy_m
        indeksy.append(obliczony_indeks)
    return indeksy
```

Rozwiązanie: Optymalizacja

- Wywołanie funkcji **tylko raz** (wynik 64-bit).
- Podział wyniku na dwie **32-bitowe** liczby.
- Kolejne indeksy obliczane z wydajnego wzoru.

Zmniejsza złożoność obliczeniową z O(k) do niemal O(1) dla hashowania.

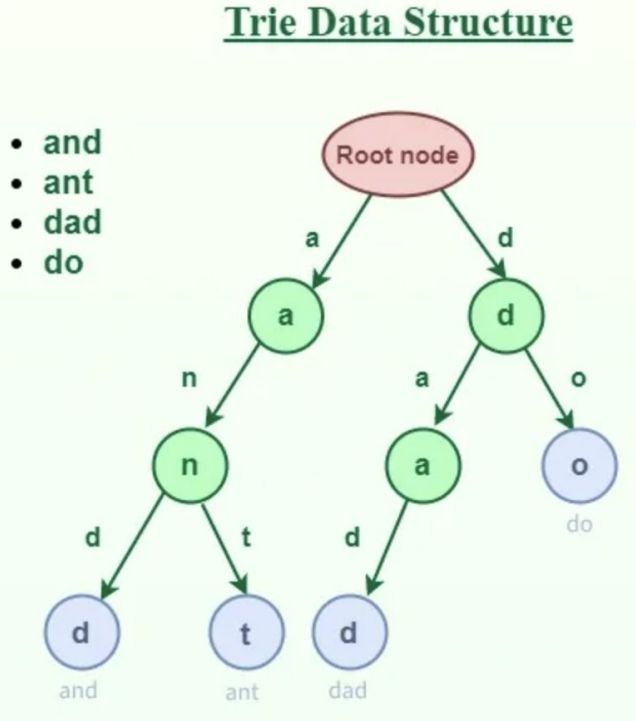
Drzewa Prefiksowe (Trie)

Charakterystyka

- Root jest pusty
- Węzeł = pojedyncza litera
- Słowo odczytywane od korzenia w dół
- Flaga końca słowa w węźle
- Złożoność: O(n) (n - długość stringa)
- Brak kolizji
- Minus: duże zużycie pamięci

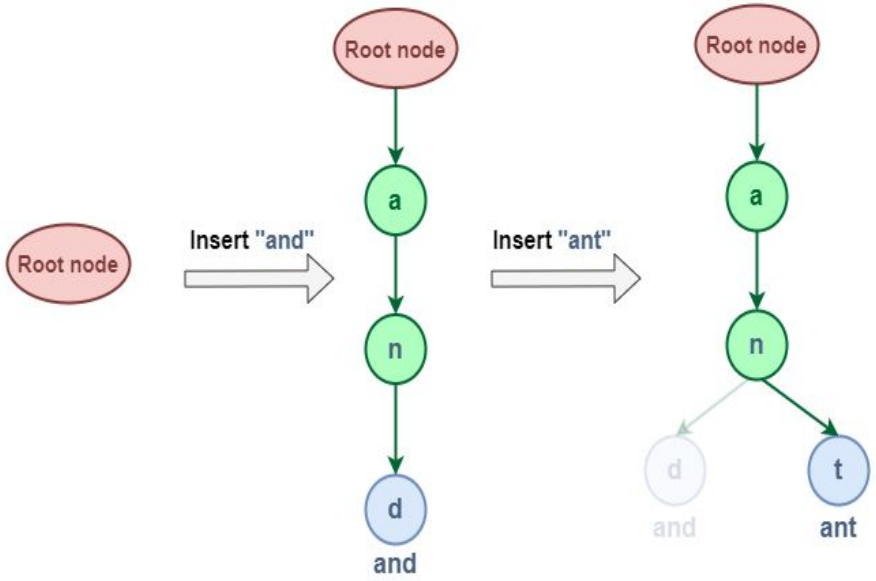
Zastosowania

- Autouzupełnianie
- Sprawdzanie pisowni
- Silniki gier (np. Scrabble)



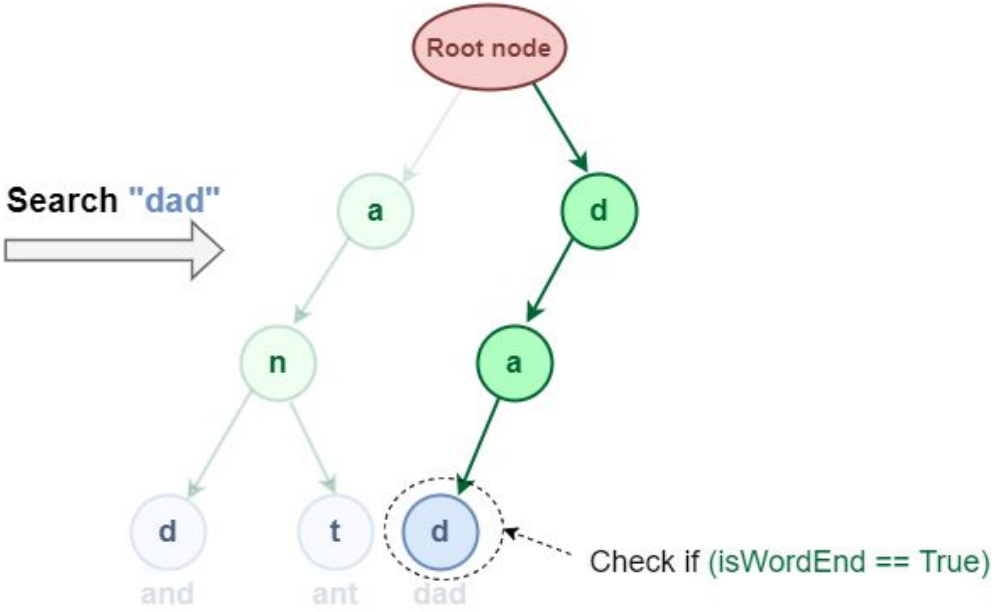
Wstawianie

```python
def insert(root, key):

    curr = root
    for c in key:
        index = ord(c) - ord('a')
        if curr.children[index] is None:
            new_node = TrieNode()
            curr.children[index] = new_node

        curr = curr.children[index]

    # Mark the end of the word
    curr.isEndOfWord = True
```



Szukanie

```python
def search(root, key):
    curr = root
    for c in key:
        index = ord(c) - ord('a')
        if curr.children[index] is None:
            return False
        curr = curr.children[index]

    # Return true if the word exists
    # and is marked as ending
    return curr.isEndOfWord
```

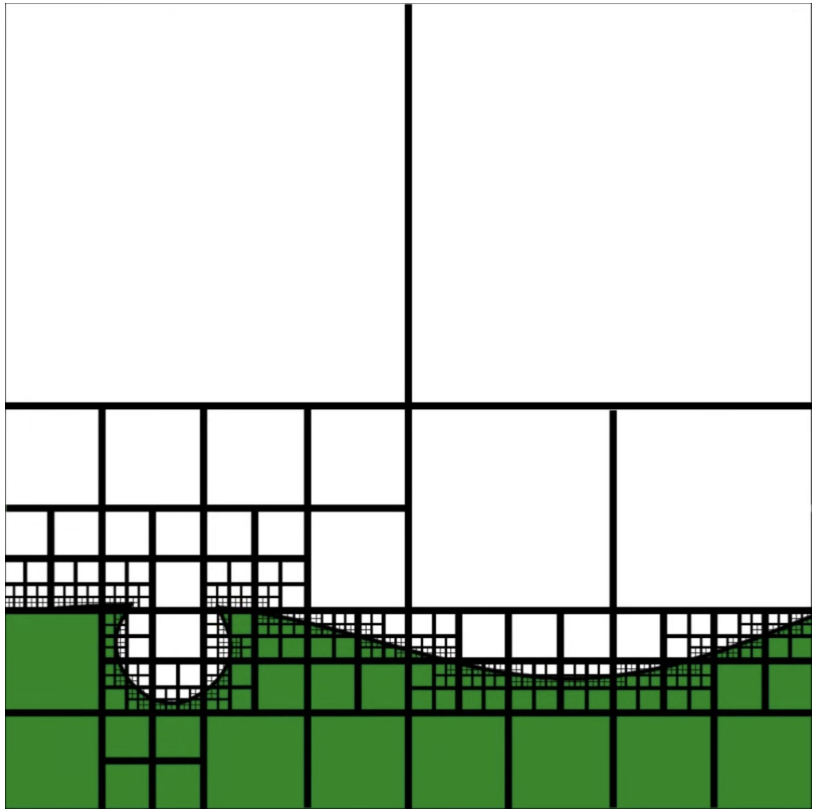


Usuwanie

Logika algorytmu

- Rekurencja: algorytm schodzi na sam dół drzewa.
- Po znalezieniu słowa, najpierw ustawiamy flagę **isEndOfWord** na **False**.
- Jeśli po odznaczeniu flagi węzeł nie posiada dzieci, zostaje on **usunięty**.
- Proces usuwania zatrzymuje się, gdy natrafimy na węzeł będący częścią innego słowa lub posiadający dzieci.

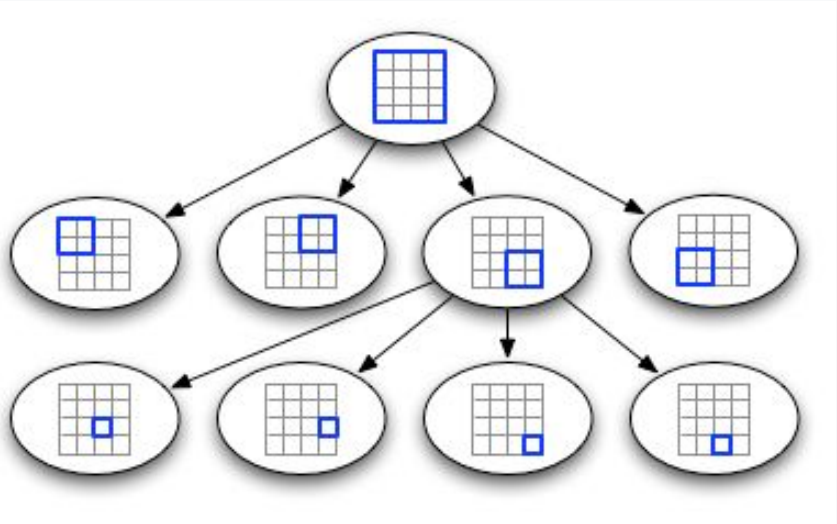
```python
def delete(self, key, curr=None, depth=0):

        if curr is None:
            curr = self.root

        if depth == len(key):
            if not curr.isEndOfWord:
                return False

            curr.isEndOfWord = False
            if all(c is None for c in curr.children):
                return True
            return False

        index = ord(key[depth]) - ord('a')
        child_node = curr.children[index]

        if child_node is None:
            return False

        should_delete_child = self.delete(key, child_node, depth + 1)

        if should_delete_child:
            curr.children[index] = None
            return not curr.isEndOfWord and all(c is None for c in curr.children)

        return False
```



Drzewa czwórkowe (quadtree)

Definicja i zastosowanie

- Struktura danych w formie drzewa, w której każdy węzeł wewnętrzny ma **dokładnie czworo dzieci**.
- Służy do organizowania i podziału **przestrzeni dwuwymiarowej** poprzez cykliczne dzielenie na 4 równe części.
- Efektywnie zarządza danymi przestrzennymi, takimi jak obrazy czy współrzędne geograficzne.



Zasady wzrostu drzewa (Quadtree)

1. Limit obiektów

Każdy węzeł posiada określony limit obiektów, które jest w stanie pomieścić.

2. Podział węzła

Przekroczenie limitu skutkuje automatycznym podziałem węzła na 4 mniejsze kwadraty.

3. Przechowywanie

Obiekty są zawsze przechowywane wyłącznie na najniższym poziomie drzewa (w liściach).

4. Struktura

Węzły znajdujące się wyżej służą jedynie do reprezentacji logicznego podziału przestrzeni.

Implementacja struktur danych

```python
class Rectangle:
def __init__(self, x, y, w, h):
self.x = x # środek
self.y = y
self.w = w # połowa szer.
self.h = h

def contains(self, point):
...

def intersects(self, other):
...
```

```python
class QuadTree:
def __init__(self, boundary, n=4):
self.boundary = boundary
self.capacity = n
self.points = []
self.divided = False

self.nw = None
self.ne = None
self.sw = None
self.se = None
```

Dzielenie na ćwiartki (Subdivision)

1. Podział przestrzeni

Metoda `subdivide()` dzieli aktualny obszar na 4 równe ćwiartki (NW, NE, SW, SE).

2. Automatyczny wzrost

Jeśli liść przekroczy limit punktów, automatycznie tworzy dzieci i rozszerza strukturę.

3. Przekazywanie danych

Nowe punkty są przesyłane do odpowiedniego węzła potomnego.

```
def subdivide(self):
x, y = self.boundary.x, self.boundary.y
w, h = self.boundary.w / 2, self.boundary.h / 2
self.nw = QuadTree(Rectangle(x - w, y - h, w, h))
# tak samo ne, sw i se
self.divided = True

def insert(self, point):
            if len(self.points) < self.capacity
      and not self.divided:
            self.points.append(point)
            return True
if not self.divided:
      self.subdivide()
return self._insert_into_children(point)
```

Faza szeroka, wąska i rekurencja

Faza szeroka

Błyskawiczne odrzucanie całych sekcji drzewa, które omijają nasz obszar poszukiwań.

Faza wąska

Precyzyjne sprawdzanie konkretnych punktów – uruchamiana tylko po dotarciu do liścia.

Rekurencja

Zautomatyzowane zejście w głąb struktury – przekazywanie zapytania do kolejnych ćwiartek, aż do znalezienia danych.

```python
def query(self, range_rect, found=None):
    if found is None: found = []

    #odrzucanie całych, nieprzecinających się gałęzi
    if not self.boundary.intersects(range_rect):
        return found

    if not self.divided:
        # 2. sprawdzanie punktów, jeśli to liść
        for p in self.points:
            if range_rect.contains(p):
                found.append(p)
    else:
        #rekurencyjne szukanie w dzieciach
        self.nw.query(range_rect, found)
        self.ne.query(range_rect, found)
        self.sw.query(range_rect, found)
        self.se.query(range_rect, found)

    return found
```

Wyszukiwanie obszarowe

Problem

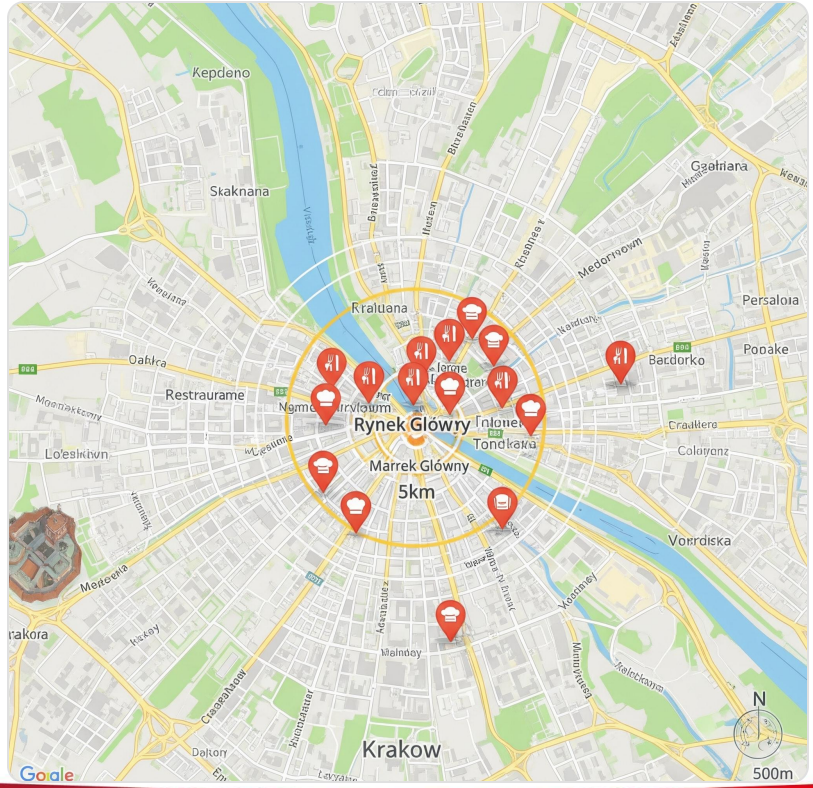
Znalezienie wszystkich restauracji w centrum Krakowa (promień 5km).

Rozwiązanie nieoptymalne

Obliczanie odległości dla każdej restauracji w Polsce i filtrowanie wyników.

Optymalne: Drzewo czwórkowe

- Sprawdzamy, czy Kraków znajduje się w danej ćwiartce.
- Jeśli tak: przechodzimy głębiej w strukturę.
- Powtarzamy do poziomu liści (np. dzielnice Krakowa).



Wady, zalety, użyteczność

Zalety

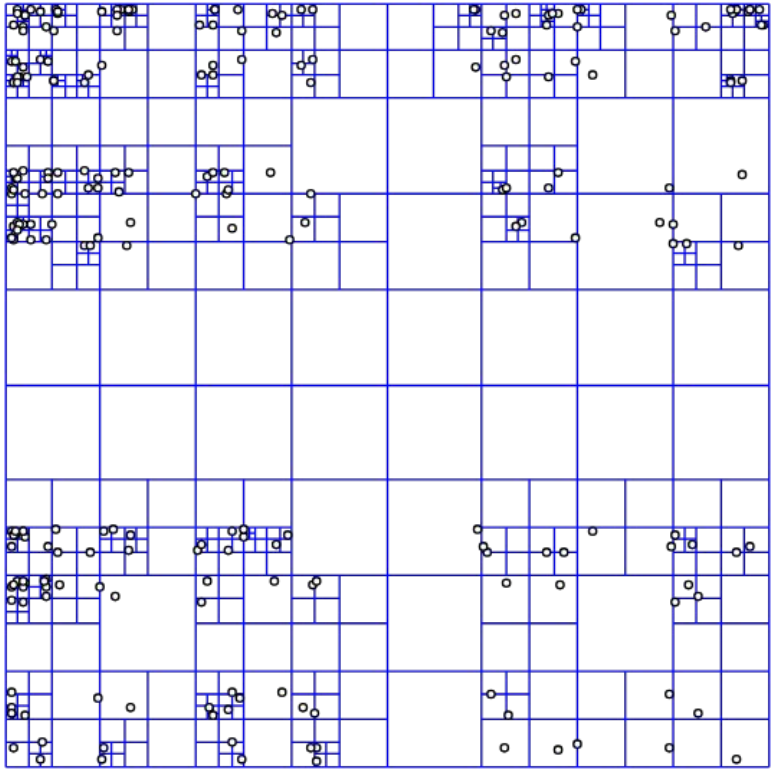
Duży skok wydajności przy zapytaniach przestrzennych:
O(log n)

Wady

Jeśli punkty skupiają się w jednym rogu, drzewo staje się bardzo głębokie i niezbalansowane.

Gdzie się używa?

- Detekcja kolizji
- Kompresja obrazu
- Renderowanie map



Drzewa skrótowe (Merkle tree)

Definicja i zastosowanie

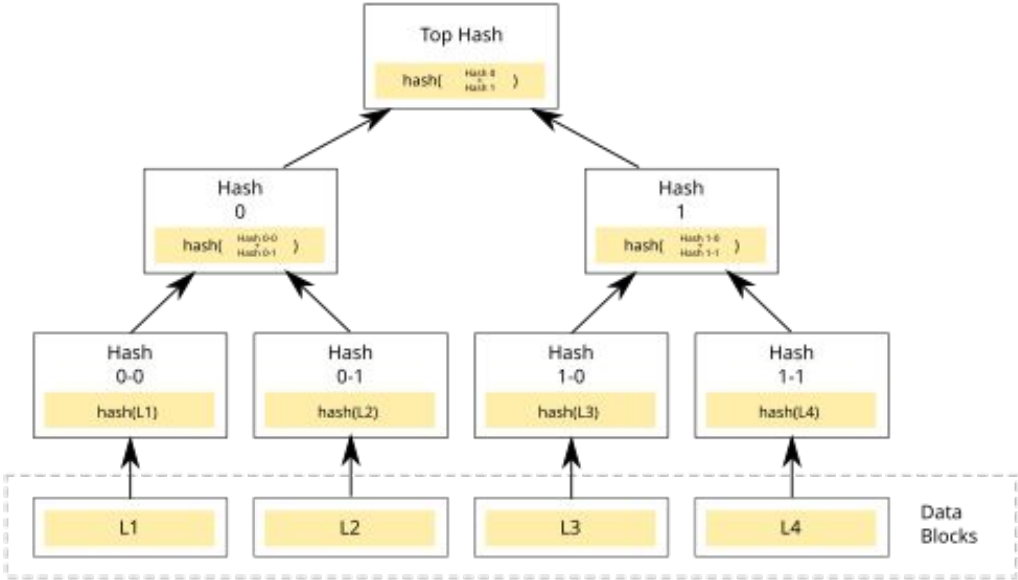
Struktura drzewiasta służąca do efektywnej weryfikacji i synchronizacji dużych zbiorów danych.

Mechanizm skrótu (Hash)

Każdy węzeł wewnętrzny to kryptograficzny hash połączonych hashy jego dzieci.

Kluczowe cechy

- Wszystkie liście na tej samej głębokości.
- Gwarancja integralności danych.
- Szybkie wykrywanie zmian.



Złożoność obliczeniowa

Przestrzeń
O(n)

Przeszukiwanie
O(log n)

Sprawdzanie integralności
O(log n)

Wstawianie
O(log n)

Usuwanie
O(log n)

Synchronizacja
O(log n)

Zastosowania

Systemy rozproszone

Wszędzie tam, gdzie te same dane muszą istnieć i być identyczne w wielu miejscach jednocześnie.

Apache Cassandra

Używane w bazach danych NoSQL do wykrywania niespójności pomiędzy replikami baz danych.

Blockchain

Fundamenty technologii stojącej za Bitcoinem i innymi kryptowalutami na całym świecie.

Struktura węzła

Referencje
Każdy węzeł przechowuje referencje do swoich dzieci.

value (Hash)
Przechowuje unikalny hash identyfikujący węzeł.

content
Połączona zawartość podelementów dla łatwiejszego śledzenia zmian.

is_copied: Flaga informująca, czy węzeł jest duplikatem.

```python
import hashlib

class Node:
    def __init__(self, left, right, value: str, content, is_copied=False) -> None:
        self.left = left
        self.right = right
        self.value = value       #hash
        self.content = content   #połączona zawartość
        self.is_copied = is_copied # flaga paddingu

    @staticmethod
    def hash(val: str) -> str:
        return hashlib.sha256(val.encode('utf-8')).hexdigest()

    def copy(self):
        return Node(self.left, self.right, self.value, self.content, True)
```

Budowa Drzewa

Zasada "Dziel i zwyciężaj"
Lista węzłów jest rekurencyjnie dzielona na połowy aż do par bazowych.

Balansowanie poziomów
Ostatni węzeł jest duplikowany metodą copy() przy nieparzystej liczbie.

Agregacja skrótów
Wartość rodzica to hash z połączenia wartości jego dzieci.

Warunek stopu
Rekurencja kończy się po uzyskaniu jednego nadrzędnego punktu.

```python
class MerkleTree:
    #  ...inicjalizacja
    def __buildTreeRec(self, nodes: list) -> Node:
        if len(nodes) % 2 == 1:
            nodes.append(nodes[-1].copy())

        half: int = len(nodes) // 2
        if len(nodes) == 2:
            return Node(nodes[0], nodes[1],
                        Node.hash(nodes[0].value + nodes[1].value),
                        nodes[0].content + "+" + nodes[1].content)


        left: Node = self.__buildTreeRec(nodes[:half])
        right: Node = self.__buildTreeRec(nodes[half:])
        value: str = Node.hash(left.value + right.value)
        content: str = f'{left.content}+{right.content}'

        return Node(left, right, value, content)
```

Bibliografia

- https://www.geeksforgeeks.org/dsa/dsa-tutorial-learn-data-structures-and-algorithms/
- https://www.youtube.com/watch?v=jxbDYxm-pXg
- https://en.wikipedia.org/wiki/Quadtree
- https://en.wikipedia.org/wiki/Merkle_tree
- https://en.wikipedia.org/wiki/Bloom_filter